UWE Bristol | University of the West of England

Presentation by

Majid Mumtaz

Software Testing

Week 11

Recap

Databases connectivity

Database Access Object

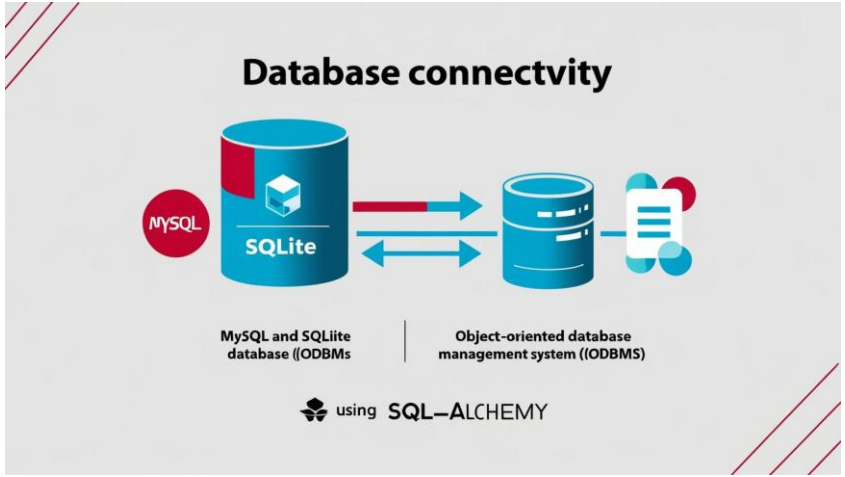
Relational Databases (SQLite and MySQL)

Classes to Relational Tables

Object Relational Mapper - SQLAlchemy

Distributed Databases

Model-View-Controller



What we will cover…

Software Testing

Why Software testing

Reasons of Software Failure

Software Quality Standards

Software Quality Evaluation

Software Quality Levels and Methods

Software Testing Demos

What and Why Software Testing…

UWE Bristol University of the West of England

What is Software Testing?

"Software testing is the act of examining the artifacts and the behaviour of the software under test by validation and verification."

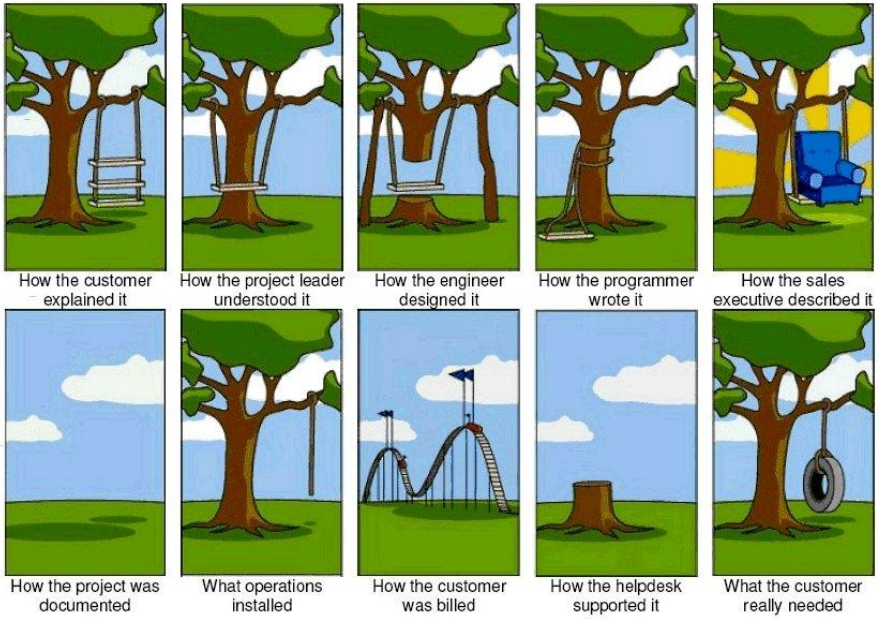
Why Software testing?

Software testing ensures that an application or system works correctly and is glitches-free.

https://testsigma.com/guides/software-testing

Reasons of Software Failure

- Unrealistic or unarticulated project goals
- Badly defined system requirements
- Inaccurate estimates of needed resources
- Poor reporting of the project's status
- Unmanaged risks
- Poor communication among customers, developers and users
- Use of immature technology
- Inability to handle the project's complexity
- Sloppy development practices
- Poor project management
- Stakeholder politics
- Commercial pressures



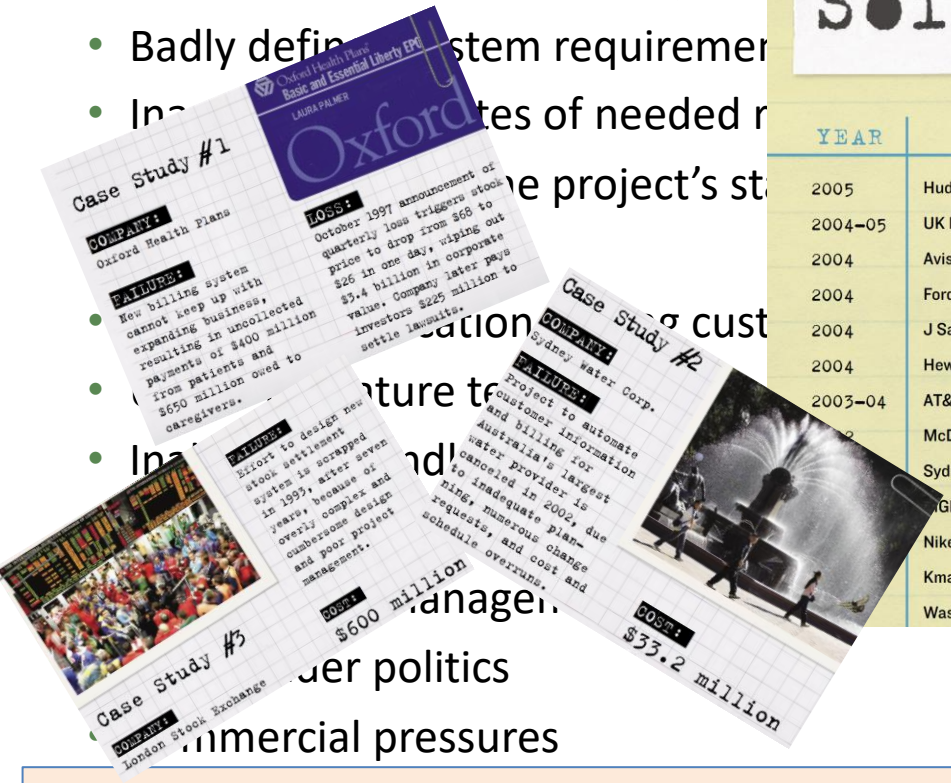
Original Image source: Unknown

Pg. 44

Source and Must Read: Robert N. Charettte, Why Software Fails, IEEE Spectrum, Sept 2005, pp. 42-49. https://ieeexplore.ieee.org/document/1502528

Reasons of Software Failure

- Unrealistic or unarticulated proje
- Badly defin stem requiremen
- In tes of needed r
- e project's st
- ation cust
- ture te
- In nd
- anager
- der politics
- mmercial pressures



Software Hall of Shame

YEAR	COMPANY	OUTCOME (COSTS IN US $)
2005	Hudson Bay Co. [Canada]	Problems with inventory system contribute to $33.3 million* loss.
2004–05	UK Inland Revenue	Software errors contribute to $3.45 billion* tax-credit overpayment.
2004	Avis Europe PLC [UK]	Enterprise resource planning (ERP) system canceled after $54.5 million† is spent.
2004	Ford Motor Co.	Purchasing system abandoned after deployment costing approximately $400 million.
2004	J Sainsbury PLC [UK]	Supply-chain management system abandoned after deployment costing $527 million.†
2004	Hewlett-Packard Co.	Problems with ERP system contribute to $160 million loss.
2003–04	AT&T Wireless	Customer relations management (CRM) upgrade problems lead to revenue loss of $100 million.
[illegible]	McDonald's Corp.	The Innovate information-purchasing system canceled after $170 million is spent.
[illegible]	Sydney Water Corp. [Australia]	Billing system canceled after $33.2 million† is spent.
[illegible]	IGNA Corp.	Problems with CRM system contribute to $445 million loss.
[illegible]	Nike Inc.	Problems with supply-chain management system contribute to $100 million loss.
[illegible]	Kmart Corp.	Supply-chain management system canceled after $130 million is spent.
[illegible]	Washington, D.C.	City payroll system abandoned after deployment costing $25 million.

Must Read: Robert N. Charettte, Why Software Fails, IEEE Spectrum, Sept 2005, pp. 42-49. https://ieeexplore.ieee.org/document/1502528

Software Quality Standards

- Software Quality Standards – **MUST READ** Source: https://www.it-cisq.org/standards/code-quality-standards/
- **ISO/IEC 25000** series of standards, known as SQuaRE (Systems and Software Quality Requirements and Evaluation)
  - **ISO/IEC 25010:2011** – System and software quality models.
    - 8 quality characteristics e.g., Functional Suitability, Performance Efficiency, Compatibility, Usability, Reliability, Security, Maintainability, Portability.
  - **ISO/IEC 25012:2008** contains a model for data quality that is complementary to 25010.
  - **ISO/IEC 25023:2016** – Measurement of system and software product quality. Software quality measures at operational or behavioural (rather than the product or source code) level and complements 25010.
  - **ISO/IEC 5055:2021** – for measuring internal structure of a software product (i.e., source code) on Security, Reliability, Performance efficiency and Maintainability.
    - Example of dangerous weaknesses: '*Allowing a path from the user interface directly to database without passing through user authentication routines*' [Architectural weakness → Security and data protection violation]
  - **ISO/IEC/IEEE 24765:2017** - system and software engineering – vocabulary
- Also read – **Quality Assurance/Management** - https://www.tutorialspoint.com/software_testing_dictionary/quality_assurance.htm

What is Evaluation?

- It is about critically examining your solution and making a judgement about its effectiveness against quality measures

  - Correctness - Does the solution solve the **actual** problem?
  - Completeness - Does your solution cover **all** aspects or requirements of the problem statement?
  - Efficiency - Are the resources (time, memory, CPU) required to run the solution/algorithm acceptable?
  - Elegance – simplicity and effectiveness of the Design/Implementation.
  - Usability - How well the can the solution be used by the end users?

- **Note**: Improving one aspect of a solution may require compromising other aspects.
  Example: System performance vs Security vs Storage

Software Testing

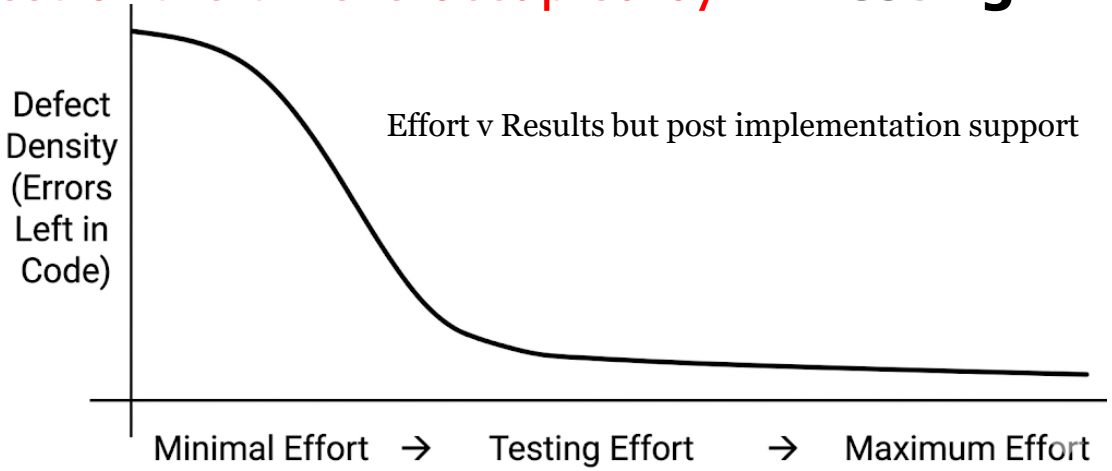
- **How do we make sure our code is error-free?**
  - Code doesn't break with incorrect or unexpected inputs
  - Code does exactly what it is supposed to do
  - Code is efficient and meets required standards

- Just imagine:
  - Can you **guarantee** your software is completely error-free?
  - Can you **guarantee** it fully meets all user requirements and needs?
  - Can you **guarantee** it is efficient and follows quality standards?
  - What if someone from the Quality Assurance team starts finding bugs in your code?

- One proven way to check and ensure software quality is → **Software Testing**

It's easy….

- … to think that software development is all about writing exciting programs….
- However, actual design and 'coding' accounts for only about half the total time taken to bring a software product to a useable state
- The rest of the time is occupied by … **Testing**



Software Testing

- **Software testing** is an *act of examining the software artifacts towards the requirements and expectations*.
- Testing is also about finding errors/bugs in a software
  - It is a *validation* and *verification* way out for the software pieces.
    - *Validation: Built the right thing* i.e. the process of evaluating software during or at the end of the development process to determine whether it satisfies specified requirements. [IEEE-24765] - https://ieeexplore.ieee.org/document/8016712
    - *Verification: Built the thing right* i.e. the process of evaluating software to determine whether the products of a given development phase satisfy the conditions imposed at the start of that phase. [IEEE-24765]
  - It is about to discover and rectify the gaps in between what is expected (required) and what is produced.

Software Testing Concepts

- *Error – Fault – Failure* is a chain triggered by a programmer, which ends up a *failure*, where developer makes an *error*, causes a *fault* in the program and ends up in a *failure* once executed.
- *Test plan* is a plan put together on how to deliver software activates implementing pre-set *test strategies.*
  - *Traceability matrix* is a tool used to link all requirements and design items to relevant implementation units so as to trace through the whole system
- *A test case defines what feature is being tested and covers details about data inputs, series of steps, expected output, associated requirement, etc.*
- A *test suit* is a collection of *test cases* collated to test the software in various respects.
- *Test data* consists of input and output data which is used in test cases. It feeds in the testing reports.
- *Test script* is a written piece of program to let repeat the action of a human tester to deliver the planned tests.
- A *test run* is a report generated on the delivery of a *test suit* or a selection of set of *test cases*.

Software Testing Concepts

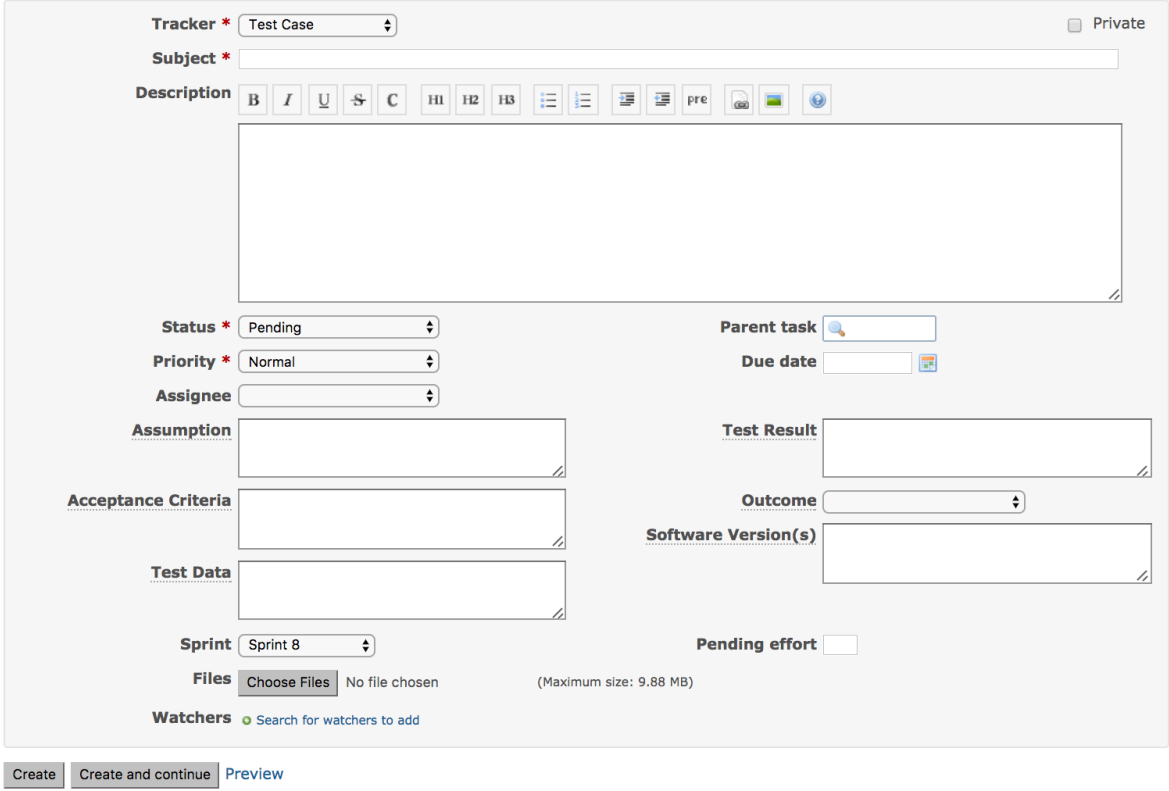
Test Case ID: [TC1.1.3]
Objective: Collaborate/provide feedback on visualisation
Function: Allow users to annotate/comment on the visualisation
Status: Draft
Testing Environment: [Specific Hardware/Software Platform]

Tested By: [Person(s) Name(s)/Automated]
Associated Req ID(s): *UC_1.3_A, FR1.1.18*
Test Type: [Unit/Integration/Pre-Acceptance/Code Review]
Software Build Version: UA M24

Seq No.	Description	Software Code [class/componentpath and action]	Test Data [If any]	Assumptions [Pre Conditions]	Expected Results [Post Conditions]	Actual Result [Pass/Fail]	Comments
Cycle 1							
1	Go to web portal and log in	Portal	-	Visualisation is exported to portal	Logged in	Pass	
2	Select the page set up with the visualisation	Portal	Vienna 3D scenario page	-	-	Pass	
3	Select "Annotation" mode from 3D Tools component	3D control portlet	-	-	A yellow sphere is shown as position indicator	Pass	
4	Click in the 3D scene to create an annotation	X3Dom portlet	-	-	A popup is shown	Pass	
5	Enter a title, some details and optionally tags	X3Dom portlet	-	-		Pass	
6	Press "Save"	X3Dom portlet	-	-	A blue cone is displayed in 3D together with a label showing the title	Pass	

Khan Z et al., FP7 UrbanAPI project – Deliverable D5.3 – Technical Evaluation. 2014

Software Testing Concepts



This interface is about a software **Redmine** for reporting Test cases.

Khan, Z and Soomro, K, (2017), H2020 Smarticipate project – Deliverable D8.1 – Evaluation Design Document. May 2017

Levels of Software Testing

- Mainly, there are three levels of testing; *unit, integration* and *system* test.

But, System testing is further extended into the following types:

- **Unit testing**: Testing individual components or modules in isolation
- **Integration testing**: Testing the interaction between integrated modules
- **Validation testing**: Ensuring the product meets user needs and expectations
- **Regression testing**: Re-testing after changes to ensure existing functionality is not broken
- **Alpha testing**: Internal testing performed by the development/QA team
- **Beta testing**: External testing performed by real users in their environment
- **Acceptance testing**: ensures the system meets user requirements. Final testing to ensure the system meets business and user requirements

Read Chapter 14 of John Hunt's book pg. 165-174

Unit Testing

- Unit testing is **automated testing** performed by developers on individual units (functions, methods, or classes) of the **source code** to ensure each **piece** works correctly in isolation.

- Unit tests verify the correctness of:
  - *Algorithms and logic, Data structures, Interfaces, Independent paths, Boundary conditions, Error handling*

- Role in Software Development: Forms the foundation of Test-Driven Development (TDD). Unit test core practice in *Extreme Programming* paradigm and *Scrum* framework.

  - Popular Frameworks such as *JUnit, xUnit family* are developed for this purpose and help out plan *Test-Driven-Development (TDD).*

  - *unittest* and *pytest* are two *Python* implementations of unit test framework.
    - unittest is the in-built python test runner
    - nose2 or nose and pytest are externally developed test runners, which require installation prior to use.
    - unittest is going to be used as part of the scope of this module

Unit-test by Wikipedia

Unit-test with Python

unittest in Python

Remember we used simple test cases for CI/CD example

main.py | main-test.py

```python
def Add(a, b):
        return a + b

def Subtract(a, b):
        return a - b

def CallHello():
        print("hello world from cicircle tutorial")

if __name__ == '__main__':
        CallHello()
```

main.py | main-test.py

```python
from main import Add, Subtract

def TestAdd():
        assert Add(2,3) == 5
        assert Add(5,5) == 10
        print("Add Function works correctly")

def TestSubtract():
        assert Subtract(5, 2) == 3
        print("Subtract Function works correctly")

if __name__ == '__main__':
        TestAdd()
        TestSubtract()
```

1. Inherit TestCase class `class TestSum(unittest.TestCase):`
2. Develop the new class with methods in which test cases are included
3. Use `.assert<>()` methods to deliver unit test

A number of `.assert()` variations are as follows:

.assert()variations

A number of `.assert()` variations are as follows:

Method	Returns
assertEqual(a, b)	a == b
assertNotEqual(a, b)	a != b
assertTrue(x)	bool(x) is True
assertFalse(x)	bool(x) is False
assertIs(a, b)	a is b
assertIsNot(a, b)	a is not b
assertIsNone(x)	x is None
assertIsNotNone(x)	x is not None
assertIn(a, b)	a in b
assertNotIn(a, b)	a not in b
assertIsInstance(a, b)	isinstance(a, b)
assertNotIsInstance(a, b)	not isinstance(a, b)

unittest with TestCase

Folder: VSCodeTest/inc_dec.py and test_unittest.py

Unittest requires implementation of `TestCase` class which is used to assure the tested functions are working correctly and producing valid results

Assume we have inc_dec.py

```python
def increment(x):
    return x + 1

def decrement(x):
    return x - 1
```

```python
import inc_dec    # The code to test
import unittest   # The test framework

class Test_TestIncrementDecrement(unittest.TestCase):
    def test_increment(self):
        self.assertEqual(inc_dec.increment(3), 4)

    def test_decrement(self):
        self.assertEqual(inc_dec.decrement(3), 4)

if __name__ == '__main__':
    unittest.main()
```

For running the test class in command line

unittestexample_functions.py -

```python
def addition_list (num, *more):
    result = num + sum(more)
    return result

def multiply_two (num1, num2):
    result = num1 * num2
    return result
```

*unittest_test.py - /Users/za2-khan/OneDrive UW

```python
import unittest
from unittestexample_functions import addition_list

class MyTest(unittest.TestCase):
    def test_sum_unknownlist_correct(self):
        self.assertEqual(addition_list(1,2) , 3)

    def test_sum_unknownlist_incorrect(self):
        self.assertEqual(addition_list(1,2) , 10)

if __name__ == '__main__':
    unittest.main()
```

unittest_test.py - /Users/za2-khan/OneDrive UWE/O

```python
import unittest
from unittestexample_functions import addition_list

class MyTest(unittest.TestCase):
    def test_sum_unknownlist_correct(self):
        self.assertEqual(addition_list(1,2) , 3)

    def test_sum_unknownlist_incorrect(self):
        self.assertEqual(addition_list(1,2,3,4) , 10)

if __name__ == '__main__':
    unittest.main()
```

```
 RESTART: /Users/za2-khan/OneDrive UWE/OneDrive - UWE Bristol (Staff)/UWE/Teaching/Web Programming All/UWE Teaching 2017-18/Learning Material/Semester 1/WK-11-20/Lecture/unittest_test.py
.F
======================================================================
FAIL: test_sum_unknownlist_incorrect (__main__.MyTest)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "/Users/za2-khan/OneDrive UWE/OneDrive - UWE Bristol (Staff)/UWE/Teaching/Web Programming All/UWE Teaching 2017-18/Learning Material/Semester 1/WK-11-20/Lecture/unittest_test.py", line 9, in test_sum_unknownlist_incorrect
    self.assertEqual(addition_list(1,2) , 10)
AssertionError: 3 != 10

----------------------------------------------------------------------
Ran 2 tests in 0.060s

FAILED (failures=1)
>>> |
```

See demo

```
PS C:\Users\m3-mumtaz\OneDrive - UWE Bristol\Spring_2026Modules\ASD Srping26\Teachi
 part\Testing\UnittestingEamples> python .\unittest_test.py
...
----------------------------------------------------------------------
Ran 3 tests in 0.000s

OK
```

UnitTest docs: https://docs.python.org/3/library/unittest.html

A **unittest** Example

```python
class Calculate:
    def __init__(self):
        pass

    def sum(self, val):
        total = 0
        for v in val:
            total += v
        return total

    def product(self, val):
        prod = 1
        for v in val:
            prod *= v
        return prod

    def add(self, n1,n2):
        return n1+n2

    def subtract(self, n1, n2):
        return n1-n2

    def multiply(self, n1, n2):
        return n1*n2

    def divide(self, n1, n2):
        return n1/n2
```

```python
from functions import *

c = Calculate()

data = [1, 2, 3, 4]

print(c.sum(data))

print(c.product(data))

print(f"{data[0]}+{data[1]}={c.add(data[0],data[1])}")

print(f"{data[2]}-{data[1]}={c.subtract(data[0],data[1])}")

print(f"{data[1]}*{data[3]}={c.multiply(data[1],data[3])}")

print(f"{data[3]}/{data[2]}={c.divide(data[3],data[2])}")
```

See the demo present in Unittesting_Example folder

```python
import unittest

from my_sum import Calculate as c

class TestSum(unittest.TestCase):
    def test_sum(self):

        d = c()
        data = [1, 2, 3]

        self.assertEqual(d.sum(data), 6)

    def test_prod(self):

        d = c()
        data = [1, 2, 3, 4]

        self.assertEqual(d.multiply(data), 24)

    def test_sum_negative(self):
        d = c()

        self.assertEqual(d.sum([1, 2, 3, -5]), -1)

if __name__ == '__main__':
    unittest.main()
```

A **unittest** Example

```python
class Calculate:
    def __init__(self):
        pass

    def sum(self,
        total = 0
        for v in v
            total
        return tot

    def product(se
        prod = 1
        for v in v
            prod *
        return pro

    def add(self, n1,n2):
        return n1+n2

    def subtract(self, n1, n2):
        return n1-n2

    def multiply(self, n1, n2):
        return n1*n2

    def divide(self, n1, n2):
        return n1/n2
```

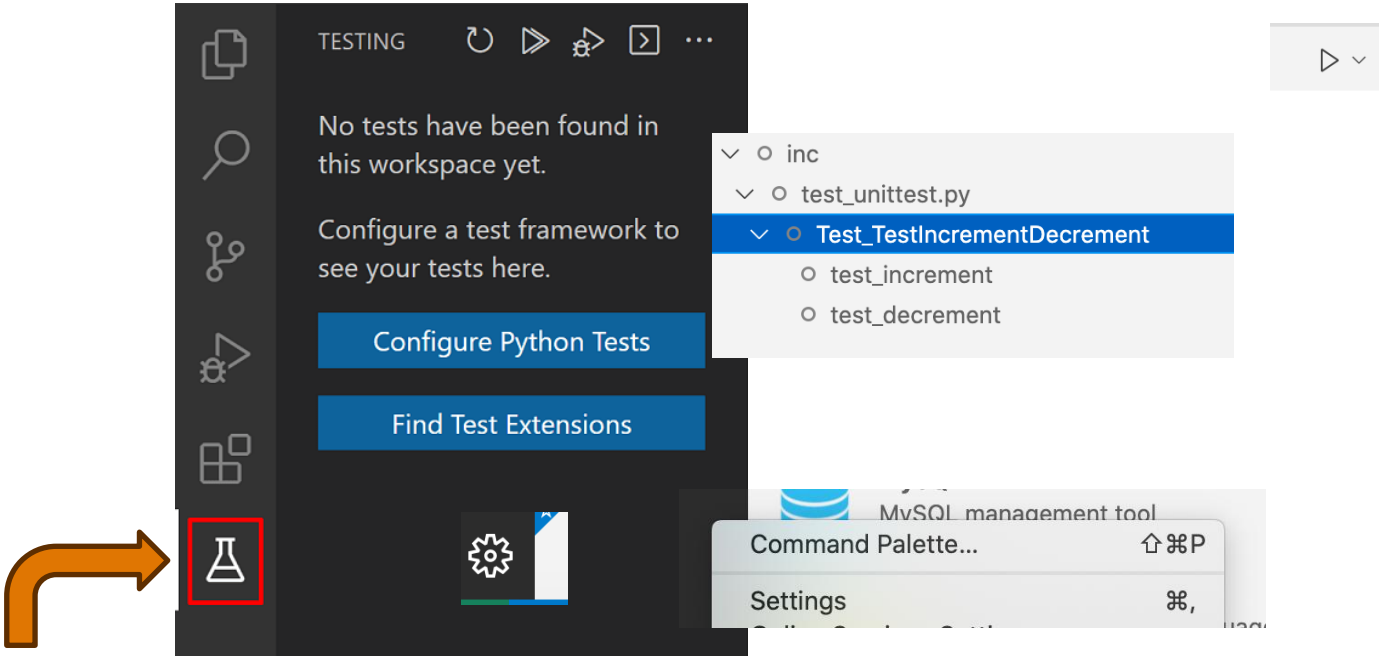
```python
from functions import *
```

```
..F
======================================================================
FAIL: test_sum_negative (__main__.TestSum)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "/Users/me-aydin/Downloads/workdr/py/others/my_sum/test_my_sum.py", line 23, in test_sum_negative
    self.assertEqual(d.sum([1, 2, 3, -5]), -1)
AssertionError: 1 != -1

----------------------------------------------------------------------
Ran 3 tests in 0.001s

FAILED (failures=1)
```

```python
print(f"{data[1]}*{data[3]}={c.multiply(data[1],data[3])}")

print(f"{data[3]}/{data[2]}={c.divide(data[3],data[2])}")
```

```python
import unittest

from my_sum import Calculate as c

class TestSum(unittest.TestCase):
    def test_sum(self):
        sum(data), 6)

        self.assertEqual(d.multiply(data), 24)

    def test_sum_negative(self):
        d = c()

        self.assertEqual(d.sum([1, 2, 3, -5]), -1)

if __name__ == '__main__':
    unittest.main()
```

VS Code settings

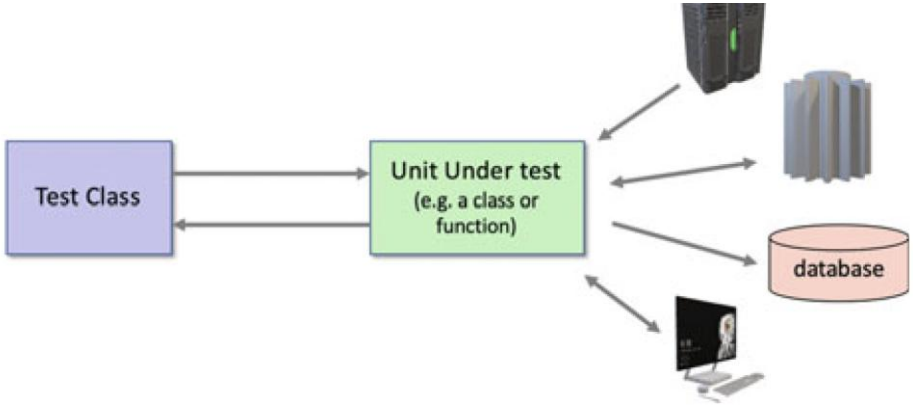


For more details on how VS Code operates [click this link](https://code.visualstudio.com/docs/python/testing)
https://code.visualstudio.com/docs/python/testing

Unit Test (Pros and Cons)

- **Pros:**
  - Helps detect problems in early development cycle
  - Useful for TDD and agile implementations
  - Helps refactor the code at the source of a fault while developing and re-test it immediately.
  - Makes easier to do next level testing (i.e. integration and systems testing) and less complex.
- **Cons:**
  - It cannot be considered as integration testing, as it is limited to the functionalities as it is automatic.
  - It is limited to selected cases, and requires too much effort to cover wider-range of possibilities
  - Realistic test cases require expertise while the developer team may not always have access to such expertise.

Unit Test (Pros and Cons)



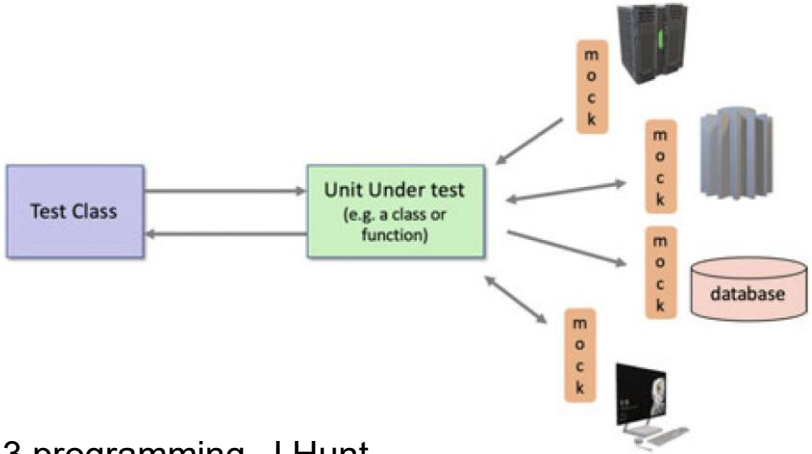


Image Source chapter 16 – Advanced Guide to Python 3 programming, J Hunt

Mock Test Example in Python

Must Read chapter 16 – Advanced Guide to Python 3 programming, J Hunt

```python
import unittest
from unittest.mock import *
from unittest import TestCase
from unittest import main

class MockTestExampleClass():
    result = 0

    # following method will be tested - magicmock
    def set_result(self):
        return 0

    def get_result(self, num):
        global result
        result = num
        return self.set_result() + num
```

```
>>>
 RESTART: /Users/za2-khan/OneDrive - UWE Bristol (Staff)/UWE/Teaching/Web Prog
ramming All/UWE Teaching 2019-20/Learning Material/WK-6 new/Lecture 2/mocktest
_magicmock_test.py
.F
======================================================================
FAIL: test_get_results_incorrect (__main__.test_MockTestExampleClass_public_in
terface)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "/Users/za2-khan/OneDrive - UWE Bristol (Staff)/UWE/Teaching/Web Progra
mming All/UWE Teaching 2019-20/Learning Material/WK-6 new/Lecture 2/mocktest_m
agicmock_test.py", line 36, in test_get_results_incorrect
    self.assertEqual(15, result, 'return value from get_result method incorrec
t')
AssertionError: 15 != 25 : return value from get_result method incorrect

----------------------------------------------------------------------
Ran 2 tests in 0.064s

FAILED (failures=1)
>>>
```

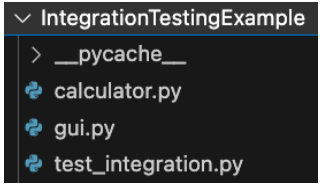
```python
class test_MockTestExampleClass_public_interface(unittest.TestCase):
    def test_get_results_correct(self):
        test_object = MockTestExampleClass()    #creating MockTestExampleClass object
        test_object.set_result = MagicMock(name = 'set_result')
        test_object.set_result.return_value = 10   #setting return value
        result = test_object.get_result(5)      #calling get_result

        self.assertEqual(15, result, 'return value from get_result method incorrect')

    def test_get_results_incorrect(self):
        test_object = MockTestExampleClass()
        test_object.set_result = MagicMock(name = 'set_result')

        test_object.set_result.return_value = 20
        result = test_object.get_result(5)

        self.assertEqual(15, result, 'return value from get_result method incorrect')

if __name__ == '__main__':
    unittest.main()
```

See demo

Integration, System and Acceptance Tests

- Integration test is about testing software once put pieces together to realise if the requirements are still met. It follows the unit test stage and conducted before system/validation tests.
  - Continuous Integration Testing – e.g. CircleCI, Jenkins; **Self study:** CI/CD Pipeline: See week 7
- It is done to evaluate the compliance of a system with respect to chosen functional requirements.
- *Validation (system) test* is the test conducted with the complete systems with a number of user stories and/or use case scenarios.
  - It is done once the integration test is completed
  - It investigates the compliances of the system with system requirements specs (SRS)
  - It uses *regression test* to see of the system complies subject to changes happened.
  - Also, *Alpha* and *Beta* testing are conducted within this context, *alpha* is conducted in a customer point of view within the developer environment, while *beta* testing is done with releasing the software to a limited number of customers
- *Acceptance test* is made by the customers themselves before releasing the software to use.
  - The end-user costumer requirements are taken on board and user-acceptance test (UAT) is conducted
  - It is similar to validation /system testing, but it is done by the customers themselves.

See demo



More concepts

- Review, walkthrough, inspection all known as *static* testing which is about **verification**
  - Testing with test cases is known as *dynamic* testing, which is about **validation**
- "Box" testing are the techniques of testing well-known with:
  - White-box and black-box testing
  - Gray box is a combination of both

Testing methods

- There are two methods of building tests:
  - "black box"
    - The core objective is a quality test
    - The evaluation of what you see?
    - How is it supposed to work?
    - What software is doing? But outwardly observing.
  - "white box"
    - examine the internal detail of units to verify that all the 'code' is functioning correctly
    - Often black box testing cannot cover everything so white box testing
    - API, code coverage, fault injection etc fall under this category
- Integration testing links white and black box testing

Black box Testing

- **“black box”**
  - the tester only tests that the outputs are correct (as per the requirements specification) for given inputs
  - This helps deliver validation of all functions (functional requirements) and check the qualities (non-functional requirements)
  - Does **booking()** of your software books the ticket/table and adds/updates database tables - without going into details of the code?
  - Date Testing in your project! For instance, no booking should be allowed for any past date!

Black-box testing techniques

http://www.tutorialspoint.com/software_engineering/software_testing_overview.htm

- **Equivalence class** - input is divided into similar classes. If one element of a class passes the test, it is assumed that all the class is passed. E.g. number of tickets to book

- **Boundary values** - The input is divided into higher and lower values. If these values pass the test, it is assumed that all values in between may pass too. Like age restriction for a certain movie

- **Cause (input) – effect (output) graphing** - In above methods, only one input value at a time is tested. In this technique combinations of input values are tested in a systematic way. E.g.

  Combination 1: Valid Tenant account, request is Emergency, Maintenance staff is Available -> System immediately send a notification to the staff and marks ticket as "Urgent to resolve".
  Combination 2: Tenant account is inactive/Terminated -> System blocks the request and displays "Contact Admin".

- **Pair-wise Testing** - In pairwise testing, the multiple parameters are tested pair-wise for their different values and system behaviour. Like (2 Bedrooms + Pet Friendly (Yes) + Furnished (No))

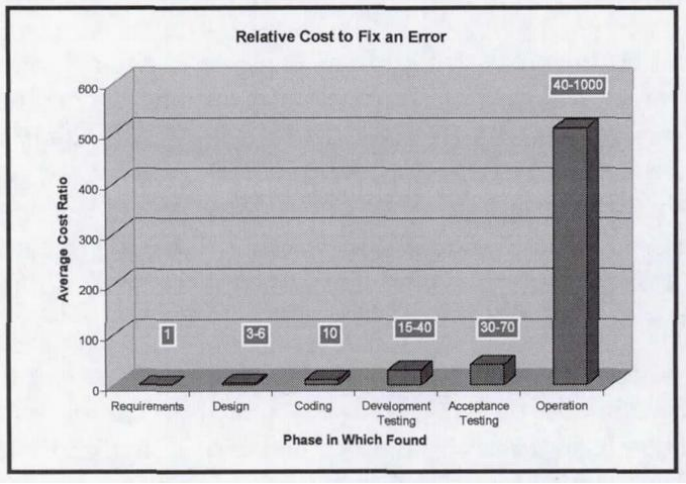
- **State-based testing** - These systems are tested based on their states and input. E.g Tracking the lifecycle of a Lease Agreement in the system.

White-box testing

- Sometimes, it becomes necessary to 'open the box' and examine the detailed program code i.e. "white-box" testing (including I/O, statements, loops, conditions etc)
- In this case, we could verify that the code is derived directly from a proven sorting algorithm - many different sorting algorithms exist
- If necessary, we could trace the execution of program statements during testing to ensure that every statement has been 'exercised'
- Then we could design tests that would specifically target statements not normally executed
- This is a kind of half-way-house between black and white box testing

White-box testing

- API, code coverage, fault injection etc fall under this category
- Again, however, the question arises of where to stop
- It has been shown that some errors in code are only revealed by a disproportionate testing effort
- Good to define **coverage matrix**
- Relative Cost to Fix Software Errors per Life Cycle Phase



Boehm 1981

Read for more statistics of defect fixing costs at different stages of SDLC: Error Cost Escalation Through the Project Life Cycle
http://ntrs.nasa.gov/archive/nasa/casi.ntrs.nasa.gov/20100036670.pdf

Axioms of testing

- The following were proposed (by Myers) many years ago and still remain a useful guide to good testing practice:
  - a good test case is one that has a high probability of detecting an error, NOT one that shows the program works correctly
  - the hardest thing is to know when to stop
  - it is impossible (or perhaps difficult) to test your own program
  - every test case must include the expected output
  - design your tests from Requirements before you start testing
  - remember to test invalid input conditions
  - thoroughly inspect the test results, preferably automatically (e.g. unit tests, Selenium - http://www.seleniumhq.org/)
  - As the number of detected bugs increases so does the probability of further bugs existing (i.e. bugs tend to cluster in software)
  - **Automate** the testing with script languages e.g. Python or other frameworks like Selenium (web)
  - ensure that system design allows for straightforward testing
  - but don't make temporary changes to code to ease testing
- Above all - know your objectives!

White-box Testing

- White-box testing involves trying to locate code which has not been run during black-box testing
- You are likely to find this difficult to do thoroughly, but you need to ensure that **places where you trap (e.g.) erroneous input values** are actually tried out
  - E.g. Check if server script received any value from client form
  - result = request.form
  - keys = list(result.keys())
  - if len(keys) > 0:
  - # means something received from client side
  - else:
  - # means nothing received from client side
- Use Try/Except/Catch blocks with meaningful messages

For Usability

- Remember user interface design principles
- User Interface
  - How easy is your system to understand?
  - How easy is your system to use e.g. user interaction?
  - Is there any help/user guide or training available?
  - Look and feel consistency
  - Consistent layout and presentation (e.g. images, contents, etc) – especially for your group project

Testing strategy

- You need to be **tough on your system during testing** - it's a good idea for someone not personally involved with the system to test it (this is the idea of Beta testing)
  - Good for groups where you can test each others code! You can also reflect on usability aspects!
- Remember that any weaknesses are likely to be exposed during the testing period! - **Testers are normally smart/experienced people** ☺
- Start testing earlier rather than later – don't leave it until your group project is complete!

Testing Goals and Requirements

- Testing goals explain why you test. ☺

Examples include finding bugs, preventing regressions, and ensuring critical flows work.

- Requirements explain what the system should do.

Example login requirements include valid email format, password length, and error handling.

- Goals guide direction.
  Requirements define behaviour.

- Turning Goals and Requirements Into Test Cases

You start with a goal.
Then you choose a requirement.
Then you write a test case.

Summary

- Software failures could be due to several reasons
- A number of standards related to software quality
- Software testing is about finding errors and having confidence in software to be reliable
- More rigorous and complete testing approach is more chances are to find errors and avoid later issues!
- Remember that ultimately someone will test and evaluate your software - even if it's the users!
- Remember cost of failure!
- Plan testing properly and test your system early in a Software Development Life Cycle
- Testing should be automated e.g., unit tests
- Testing plays an important role in CI/CD

References

UWE Bristol University of the West of England

Software quality and Testing

Sources:

1) Robert N. Charettte, Why Software Fails, IEEE Spectrum, Sept 2005, https://ieeexplore.ieee.org/document/1502528

2) Chapter 23 Software Testing from Software Engineering pg. 205-233 8 Edition by Ian Sommerville [Online] – Software Testing Overview: http://www.tutorialspoint.com/software_engineering/software_testing_overview.htm

3) Chapter 24 Error and Exception Handling from Beginners Guide to Python 3 by John Hunt

4) Part III Testing - Chapter 14, 15, 16 from Advanced Guide to Python 3 Programming by John Hunt

5) RealPython Tutorial: Getting started with Python Testing – unit testing and integration testing. https://realpython.com/python-testing/#unit-tests-vs-integration-tests

6) Top Trends of Software testing https://testsigma.com/blog/top-software-testing-trends-to-look-out-for-in-2022/